

对象存储系统设计：一次完整的推演记录

题源： Coupang Seattle L6-2 面经中出现的系统设计题（design blob storage / S3）。

题面： 设计一个对象存储服务。用户可以上传、下载、删除任意大小的文件。要求高可用、高持久性。

1. 约束

项	值
用户数	1000 万（企业租户为主）
上传峰值	5 万 QPS
下载峰值	50 万 QPS（读写比 10:1）
对象大小分布	80% < 1MB，15% 1–100MB，5% > 100MB，最大 5TB
总存储量	10 EB
持久性	11 个 9
可用性	99.99%

2. 最该问的澄清点：不是 SLO，是对象数量

一般的系统设计题先问 SLO。这道题不同——**最敏感的参数是「平均对象大小」**，因为它决定元数据规模，而元数据规模决定了一半的机器数。

同样是 10 EB：

平均对象大小	对象数	目录元数据	元数据机器（×3 副本）
0.5 MB	2 万亿	3320 TB	49,800
2 MB	5000 亿	830 TB	12,450
28.4 MB	3521 亿	58.5 TB	877
100 MB	1000 亿	16.6 TB	249

跨 200 倍。 总容量只决定 chunkserver 数量，平均对象大小决定元数据服务规模——**两个数字必须都问到**，只拿到 10 EB 是不够的。

从分布反推对象数

题面给了大小分布，可以正向推出对象数：

加权平均 = 0.80×0.5MB + 0.15×20MB + 0.05×500MB = 28.4 MB
对象数 N = 10 EB / 28.4 MB = 3521 亿

档	占比	代表大小	对象数	占用容量
< 1MB	80%	0.5MB	2817 亿	0.14 EB
1–100MB	15%	20MB	528 亿	1.06 EB
> 100MB	5%	500MB	176 亿	8.80 EB

注意最后一列：5% 的大对象占了 88% 的容量，80% 的小对象只占 1.4%。 这个长尾结构决定了后面所有的取舍——小对象贡献元数据压力，大对象贡献存储压力。

3. 分块：传输块与存储块是两回事

两个不同的约束，必须分开定：

块类型	大小	约束来源
传输块	2 MB	重传成本——失败时只需重传这一片
存储块（chunk）	最大 64 MB，变长	元数据量——块越大，元数据条数越少

变长是关键。若 chunk 是固定 64MB 的分配单位，那 2817 亿个小对象各占一个 64MB 槽位：分配 0.3 EB，实际用 0.002 EB，**浪费 99%**。变长分配（一个 0.5MB 的对象就是一个 0.5MB 的 chunk）则没有浪费。GFS 就是这么做的——chunk 是 chunkserver 上的一个普通文件，懒分配。

块数：

档	每对象块数	块数
< 1MB	1	2817 亿
1-100MB	1	528 亿
> 100MB	8	1408 亿
合计		4754 亿

一个被否决的方案：把多个小对象打包进同一个 chunk。这在「chunk 固定 64MB」的前提下能省 128 倍空间，但在变长 chunk 下没有必要，反而引入删除时的碎片整理问题。**前提变了，结论就变了。**

4. 两层元数据

底层 chunk 与元数据，上层 directory——和文件系统的 inode / 目录项是同一个结构。

4.1 目录层：object_key → [chunk_id...]

档	对象数	chunk/对象	key	chunk_id	每条	小计
< 1MB	2817 亿	1	100 B	8 B	108 B	30.4 TB
1-100MB	528 亿	1	100 B	8 B	108 B	5.7 TB
> 100MB	176 亿	8	100 B	64 B	164 B	2.9 TB
合计	3521 亿					39.0 TB

小对象贡献了 78% 的目录表大小，而它们只占 1.4% 的数据量。

key 长度是主要变量：

key 平均长度	目录表	机器数（200GB × 3 副本）
32 B	15.1 TB	226
64 B	26.3 TB	395
100 B	39.0 TB	585

4.2 块层：chunk_id → 位置

每条元数据的字段：

(machine_id 8 + disk_id 8) × 3 副本	= 48 B
chunk_id	= 8 B (全局唯一, 只存一次)
校验和 4 × 3	= 12 B
长度 4 + 版本 4 + 状态 1 + 创建 8	= 17 B
	85 B

4754 亿 × 85 B = **40.4 TB**

不存修改时间。 对象存储是不可变的——改对象等于写新对象，块写完就冻结。这省下 8 字节 × 4754 亿 = 3.8 TB，同时也简化了副本一致性（写完即冻结，不存在并发修改）和校验和管理（一次算完永久有效）。

4.3 为什么存 machine_id 而不是 IP

方案	代价
存 machine_id，另有一张小表 machine_id → IP	多一层查表，但小表只有几万条，可全内存、所有网关缓存
直接存 IP	省一跳，但任何机器变更都要改海量元数据记录

几万台的规模下每天都有机器故障、替换、扩容。**多的那一层查表在内存里几乎零成本，换来的是元数据的稳定性。** 选前者。

4.4 元数据总量

层	大小	单份机器	3 副本
目录层	39.0 TB	195	585
块层	40.4 TB	202	606
合计	79.4 TB	397	1,191

5. chunk_id 的设计

5.1 全局唯一 vs 机器局部

选全局唯一。 三个理由：

- 1. **元数据小 20%** —— 三副本共用一个 ID，不用各存一份，省 2.5 TB
- 2. **副本迁移不改 ID** —— 块从机器 A 搬到 B，元数据只改 machine_id
- 3. **GC 与对账简单** —— 直接比对 chunkserver 上的 ID 集合与元数据里的引用集合

中心化分配的代价接近零， 只要用号段租约：

每次领取	分配器 QPS
1,000	50
100,000	0.5
1,000,000	0.05

峰值 5 万次写/秒，网关一次领 10 万个号，分配器每两秒才被调用一次。分配器挂了，各网关手上的号段还能用很久。

5.2 顺序 ID vs 内容哈希

内容哈希（CAS）天然去重，但有五个代价：

- 1. **ID 变长** —— 256 位比 8 字节大 4 倍，额外 3.7 TB 元数据
- 2. **写前必须先查** —— 每次写入多一跳

- 3. 需要引用计数 —— 一个块被多对象引用，删除要等计数归零
- 4. 加密后失效 —— 客户端加密上传时，相同内容哈希不同，去重完全没用
- 5. 数据分布不可控 —— 内容决定 ID，ID 决定分片，无法做冷热分离或地理放置

结论：主路径用 int64 fingerprint 顺序 ID；若需去重，另建一张 content_hash → chunk_id 表，只在需要时查。

顺带：64 位空间下 4754 亿个块，生日碰撞概率约 0.05%。真实系统会做碰撞检测或用更宽的 ID。

6. 存储层：机器数由容量决定

6.1 一次被数字否决的配置

若按 20 TB/台 + 3 副本：

$$10\text{ EB} \times 3 = 30\text{ EB 物理容量}$$
$$30\text{ EB} \div 20\text{ TB} = 150\text{ 万台}$$

算术正确，但 20 TB/台 相当于一台机器只挂一块盘。这个配置下每台每秒只有 0.2 次读、0.4 MB/s——买了 150 万台机器，每台只用它千分之一的能力。

6.2 两个杠杆

密集存储机型：

机型	容量
常规 2U（12–24 盘 × 20TB）	250–500 TB
高密度 4U JBOD（60–102 盘）	1.2–2.4 PB

纠删码替代副本：

方案	冗余系数	容忍故障数	恢复代价
3 副本	3.0	2	低（直接拷贝）
RS(6,3)	1.5	3	高（读 6 块重算）
RS(10,4)	1.4	4	高

纠删码用更少的存储换更强的容错，代价是恢复时要读多块做计算、读取延迟略高。

GFS 的”廉价机器”常被误读。它指的是相对当年 EMC/NetApp 那类专用存储阵列，用普通 x86 + 普通 SATA 盘靠软件做冗余，**不是指每台只挂一块盘**。而且 GFS 是 2003 年面向 PB 级的设计；Google 自己的继任者 Colossus 用的正是纠删码——EB 级规模下 3 副本的成本无法接受。

6.3 结果

每台容量	3 副本	纠删码 10+4
20 TB	150 万	70 万
0.5 PB	6 万	2.8 万
2 PB	1.5 万	7 千

取 0.5 PB/台 + RS(10,4) → 2.8 万台 chunkserver。

6.4 压缩没有想象中有用

数据类型	压缩率
文本、日志、JSON	3-10×
图片（JPEG/PNG）	1.0×
视频（H.264/H.265）	1.0×
已加密数据	1.0×

对象存储的实际构成以图片、视频、备份为主，而图片视频**已经是压缩格式**。整体压缩率通常只有 **1.2-1.5×**，不是 5×。估算时按 1.2-1.5 更稳。

7. 读写路径：控制面走网关，数据面直连

7.1 为什么必须直连

方案	网关机器数
网关代理全部数据流	176 台 （仅带宽，100Gbps 网卡）
客户端直连 chunkserver，网关只发令牌	几台

代理方案里每字节要过网关两次（进 + 出），1.1 TB/s 的流量就需要 176 台纯转发机器。直连的话网关只处理控制流（每请求约 1KB），0.55 GB/s，几台就够。

GFS 和 S3 都是这个模式。网关返回（chunkserver 地址，chunk_id，有效期令牌），客户端拿令牌直接读写。

7.2 读路径

```
client
  → 网关（按 object_key hash 路由）
  → 目录元数据：key → [chunk_id...]
  → 块元数据（按 chunk_id hash 路由）：chunk_id → [(machine_id, disk_id)] × 3
  → 直连 chunkserver 读数据
```

两跳元数据 + 一跳数据，每跳内存查询亚毫秒。

一个可选优化：小对象（80%）只有一个 chunk，两跳元数据对它们是纯开销。可把单块对象的位置内联进目录表，省掉第二跳。代价是目录表每条多 48 字节（3 副本位置），约 13.5 TB 内存。用 13 TB 换 80% 请求少一跳——值不值得看延迟预算。

7.3 写路径与跨系统时序

写路径对称，但多一个跨系统一致性问题：数据写入 chunkserver 和元数据登记不能原子完成。

顺序	失败后果
先写元数据，再写数据	元数据指向不存在的块 → 读取报错
先写数据，再写元数据	块存在但无人引用 → 孤儿块，占空间

选后者。孤儿块可由后台 GC 扫描回收（比对 chunkserver 上的块与元数据里的引用），而悬空元数据会直接导致读失败。

7.4 分片上传与断点续传

大对象分多次 2MB 上传，需要：

- 上传会话（multipart upload）**——客户端先 initiate 拿 upload_id，每片带（upload_id, part_number），最后 complete 提交
- 分片状态存在会话元数据里**（哪些片已收到、每片校验和），不存在 chunkserver 上。客户端断线重连后查询即可续传
- 未完成会话要有 TTL**——用户传一半离开，那些分片是垃圾，超时（如 7 天）自动清理

错误处理：

- 单片失败 → 重传该片，按 `part_number` 幂等覆盖
- 中途断线 → 会话仍在，查已上传片继续
- `complete` 时校验所有片齐全、校验和匹配
- 令牌过期 → 回网关换新令牌

7.5 读写粒度不对称

	粒度	理由
写	2 MB	失败重传成本低
读	更大（如 24 MB）	摊薄寻道开销

HDD 一次寻道约 10ms：

读大小	寻道 + 传输	有效吞吐
2 MB	10ms + 10ms	100 MB/s
24 MB	10ms + 120ms	185 MB/s

写的粒度由重传成本决定，读的粒度由吞吐效率决定。这是读写不对称的合理设计。

8. QPS 校验：这个系统是容量驱动的

元数据层

层	机器数	读 QPS/台	写 QPS/台
目录服务	585	855	85
块元数据	606	825	82

内存 KV 单机能做 10 万-100 万 QPS，用了不到 1%。

Chunkserver 层（2.8 万台）

指标	每台	上限	占用
读请求	17.9 req/s	—	—
读带宽	35.7 MB/s	12.5 GB/s（100Gbps）	0.29%
写带宽	3.6 MB/s	12.5 GB/s	0.03%
磁盘 IOPS	0.71/盘	150/盘（HDD）	0.48%

三个维度都是千分之几到百分之几。

口径提醒：「50 万 QPS」指请求数还是对象数，差 14 倍（每对象平均 14 个 2MB 分片）。带宽才是不变量——2MB × 50 万 和 24MB × 4.2 万 都是 1 TB/s，每台负载完全相同。面试中要问清口径，但结论不受影响。

结论：整个系统由容量驱动，吞吐远不是瓶颈。QPS 涨 10 倍机器数都不用变，直到带宽或 IOPS 打满——大约是现在的 20 倍流量。

这意味着设计的主要约束是元数据规模和存储成本，不是吞吐。

9. CDN：为什么这类系统一定要有

正向推导 YouTube 的量级：

DAU 25 亿 × 每人每天 1 小时 = 25 亿小时/天 = 9 万亿秒/天
切片 5 秒一段 → 1.8 万亿切片/天
边缘 QPS 均值 = 2083 万, 峰值 3 倍 = 6250 万

带宽交叉验证（3 Mbps 720p）：单切片 1.88 MB × 2083 万 = **39 TB/s**，与按码率直算一致。

CDN 之后源站承受的：

CDN 命中率	源站 QPS	源站带宽
90%	208 万	3.9 TB/s
95%	104 万	2.0 TB/s
99%	20.8 万	0.39 TB/s

边缘 2083 万 QPS，源站 100 万——差 20 倍，中间那一层就是 CDN。

CDN 有效的三个条件，对象存储全部满足：

1. **内容不可变** —— 写完不改，缓存永不失效
2. **热点集中** —— 少数内容占绝大部分访问
3. **可容忍分发延迟** —— 新内容传到边缘需要时间

所以视频平台和对象存储的骨架是同一个：**CDN 扛读，源站扛写和长尾**。本设计中若 50 万读 QPS 里 80-90% 是热对象，源站实际只需扛 5-10 万。

10. 设计要点回顾

1. **这道题最该问的不是 SLO，是对象数量**。平均对象大小让元数据规模跨 200 倍，而总容量只决定 chunkserver 数——两个数字都要问到。
2. **传输块与存储块是两个不同的约束**。2MB 由重传成本决定，64MB 由元数据量决定。混为一谈会得出错误的分块大小。
3. **chunk 变长是关键**。固定 64MB 分配会让 2817 亿个小对象浪费 99% 空间；变长则完全无浪费。**前提变了，“打包小对象”这个优化就不必要了**。
4. **长尾结构决定取舍**：5% 的大对象占 88% 容量（存储压力），80% 的小对象占 78% 目录元数据（元数据压力）。两类压力落在不同的组件上。
5. **元数据存 machine_id 而非 IP**。多一层内存查表几乎零成本，换来的是几万台规模下机器变更不必改动海量元数据。
6. **数据面必须直连**。网关代理全部流量需要 176 台纯转发机器；只发令牌则几台就够。控制面与数据面分离是这类系统的通用模式。
7. **跨系统写入选“先数据后元数据”**。孤儿块可 GC 回收，悬空元数据会直接导致读失败。
8. **容量驱动的系统，QPS 校验是为了确认它不是瓶颈**，而不是为了算机器数。本设计三个维度都只用了个位数百分比。
9. **密集机型 + 纠删码是 EB 级的必选项**。20TB/台 + 3 副本要 150 万台，0.5PB/台 + RS(10,4) 只要 2.8 万台——差 50 倍。
10. **压缩对对象存储几乎无效**（图片视频已压缩），估算时用 1.2-1.5× 而非 3-5×。